

Rapport de Projet - SuperHero Manager

1. Présentation générale

Le projet **SuperHero Manager** a pour objectif de développer une application web complète (Full-Stack) permettant la gestion d'une base de données de super-héros. Initialement basé sur un fichier JSON brut, le projet a été transformé en une véritable application client-serveur dynamique.

L'application permet non seulement la consultation (recherche, filtres par univers, détails d'un héros) mais intègre également un système d'authentification robuste et des opérations de modification (CRUD complet : Création, Lecture, Mise à jour, Suppression) associées à un système de rôles (Administrateur, Éditeur, Visiteur).

Le projet a été pensé pour offrir une expérience utilisateur (UX) fluide, rapide, et une interface premium et moderne, démontrant la puissance de la stack "MERN" (MongoDB, Express, React, Node.js) enrichie avec TypeScript.

2. Architecture et technologies

Le projet repose sur une architecture moderne de type "Single Page Application" (SPA) connectée à une API RESTful centralisée. L'ensemble du code est typé avec **TypeScript** pour garantir la stabilité.

Back-end (Serveur API)

- **Environnement** : Node.js avec Express.js pour construire l'API.
- **Base de données** : MongoDB, utilisé pour sa flexibilité avec les documents JSON.
- **ORM / ODM** : Mongoose, pour modéliser les schémas stricts des héros et des utilisateurs.
- **Sécurité** : JWT (JSON Web Tokens) pour sécuriser les requêtes, et bcrypt pour le hachage des mots de passe.
- **Fichiers** : Multer pour l'upload et le stockage local des images associées aux super-héros.

Front-end (Interface Utilisateur)

- **Cœur** : React.js, permettant un rendu composant par composant extrêmement rapide.
 - **Outils** : Vite, utilisé comme "bundler" pour sa rapidité de compilation.
 - **Routage** : React Router DOM pour gérer la navigation sans recharger la page.
 - **Appels Réseaux** : Axios, configuré avec des intercepteurs pour injecter automatiquement le token JWT.
 - **Formulaires & Validation** : Formik associé à Yup pour gérer facilement l'état des formulaires complexes (ajouts de héros) et les validations.
 - **Design** : CSS natif (Vanilla CSS) exploitant des concepts modernes comme le Glassmorphism, le Flexbox/CSS Grid, et des animations subtiles pour un rendu "Premium".
-

3. Fonctionnalités réalisées

Toutes les fonctionnalités demandées dans le cahier des charges ont été implémentées :

1. **Système d'Authentification** : Inscription et connexion avec gestion de token JWT conservé dans le `localStorage`.
2. **Routage Protégé et Rôles** : L'interface et les routes API sont protégées selon le rôle (`admin` et `editor` peuvent modifier/ajouter, seul `admin` peut supprimer).
3. **Tableau de Bord** : Affichage esthétique sous forme de cartes.

4. **Filtres et Tri** : Possibilité de filtrer les héros par univers (Marvel, DC, etc.), rechercher par nom, et trier de A à Z ou par date d'ajout.
 5. **Détail du Héros** : Page complète affichant les statistiques de puissance via des barres de progression graphiques.
 6. **CRUD complet** : Ajout et Édition via un formulaire sécurisé (Formik/Yup). Suppression sécurisée.
 7. **Gestion des Images** : L'upload de nouvelles images via Multer fonctionne parfaitement ; ces images sont prioritaires sur les liens externes de la base JSON initiale.
 8. **Script d'initialisation** : Un script de "seeding" (`npm run seed`) a été développé pour formater et insérer les milliers de héros du JSON initial dans MongoDB.
-

4. Difficultés rencontrées

- **Conflits de Ports (macOS)** : Le port par défaut standard (5000) de l'API était initialement intercepté par le processus natif d'Apple "AirPlay Receiver" sur macOS. Il a fallu analyser le trafic et migrer le serveur Node sur le port `5001` .
 - **Migration vers TypeScript Strict** : L'adaptation du projet Front-end (initialement en Vanilla TS avec Three.js) vers React a nécessité une configuration fine de `vite-env.d.ts` et du `tsconfig.json` (notamment la gestion du JSX et la résolution des modules CSS) pour éviter les erreurs de compilation.
 - **Hameçonnage des middlewares Mongoose** : Gérer correctement le contexte (`this`) et le hachage des mots de passe dans les hooks "pre-save" de Mongoose sous TypeScript demande de se détacher des anciens callbacks `next()` pour utiliser des promesses asynchrones propres.
-

5. Axes d'amélioration

S'il fallait faire évoluer le projet, plusieurs points pourraient être approfondis :

1. **Stockage Cloud des Images** : Au lieu de stocker les images téléchargées dans le dossier local `/uploads` du serveur, il serait pertinent de configurer un bucket Amazon S3 ou un Cloudinary pour la production.
2. **Journalisation et Audit (Bonus partiel)** : Ajouter un modèle `Log` en base de données qui enregistre "qui a modifié quel héros à quelle heure", et l'exposer sur une page d'administration.
3. **Pagination** : Le JSON initial contient des centaines de héros. Bien que le rendu React soit rapide, implémenter une véritable pagination (ou un "Infinite Scroll") côté API (avec `.skip()` et `.limit()` de Mongoose) améliorerait drastiquement les performances réseau sur les connexions lentes.
4. **Déploiement Complet (Docker / VPS)** : Fournir un `docker-compose.yml` final incluant le Front-end (via Nginx), l'API Node, et la base MongoDB pour le mettre en ligne de façon autonome sur un VPS.